

Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

la

Lucrarea de laborator Nr. 4

Disciplina: Tehnici de programare

Tema: "learnpython.org"

A efectuat :

st. gr.SI – 262 Caragia Denis

A verificat :

profesor universitar Marius Dumitrașcu

Chișinău – 2026

SARCINA LUCRĂRII

Scopul lucrării este de a consolida cunoștințele de programare în limbajul Python prin rezolvarea exercițiilor practice de pe platforma LearnPython.org.

Desfășurarea lucrării

În cadrul lucrării au fost parcurse și rezolvate exerciții referitoare la principalele elemente ale limbajului Python:

- variabile și tipuri de date;
- liste și operatori;
- lucrul cu șiruri de caractere;
- instrucțiuni condiționale;
- cicluri for și while;
- funcții;
- clase și obiecte;
- dicționare;
- module;
- lucrul cu fișiere;
- generatoare și comprehensiuni.

Exercițiile au fost realizate direct pe platformă, verificând corectitudinea soluțiilor prin rularea codului.

CONCLUZIE

În urma realizării lucrării, am aprofundat noțiunile de bază ale limbajului Python și am exersat utilizarea acestora în probleme practice. Exercițiile de pe LearnPython.org au contribuit la dezvoltarea abilităților de scriere și verificare a programelor Python.

script.py

```
1 print("Hello, World!")
```

IPython Shell

```
<script.py> output:
Hello, World!

In [1]:
```

Great job!

Solution

Submit

script.py

```
1 # change this code
2 mystring = "hello"
3 myfloat = 10.0
4 myint = 20
5
6 # testing code
7 if mystring == "hello":
8     print("String: %s" % mystring)
9 if isinstance(myfloat, float) and myfloat == 10.0:
10     print("Float: %f" % myfloat)
11 if isinstance(myint, int) and myint == 20:
12     print("Integer: %d" % myint)
```

IPython Shell

```
<script.py> output:
String: hello

<script.py> output:
String: hello
Float: 10.000000
Integer: 20

In [1]:
```

Great job!

Solution

Submit

script.py

```
1 numbers = []
2 strings = []
3 names = ["John", "Eric", "Jessica"]
4
5 # write your code here
6 second_name = None
7 numbers.append(1)
8 numbers.append(2)
9 numbers.append(3)
10 strings.append("hello")
11 strings.append("world")
12 second_name = names[1]
13 # this code should write out the filled arrays and
14 # the second name in the names list (Eric).
15 print(numbers)
16 print(strings)
```

IPython Shell

```
Traceback (most recent call last):
  File "script.py", line 7, in <module>
    numbers.append(1,2,3)
TypeError: append() takes exactly one argument (3 given)

<script.py> output:
[1, 2, 3]
['hello', 'world']
The second name on the names list is Eric

In [1]:
```

Great job!

Solution

Submit

script.py

```
1 x = object()
2 y = object()
3
4 # TODO: change this code
5 x_list = [x] * 10
6 y_list = [y] * 10
7 big_list = x_list + y_list
8
9 print("x_list contains %d objects" % len(x_list))
10 print("y_list contains %d objects" % len(y_list))
11 print("big_list contains %d objects" % len
    (big_list))
12 # testing code
13 if x_list.count(x) == 10 and y_list.count(y) ==
    10:
14     print("Almost there...")
```

IPython Shell

```
<script.py> output:
x_list contains 1 objects
y_list contains 1 objects
big_list contains 2 objects
Great!

<script.py> output:
x_list contains 10 objects
y_list contains 10 objects
big_list contains 20 objects
Almost there...
Great!
```

Good work!

Solution

Submit

script.py

```
1 data = ("John", "Doe", 53.44)
2 format_string = "Hello %s %s. Your current balance
  is %s."
3
4 print(format_string % data)
```

IPython Shell

```
<script.py> output:
Hello John Doe. Your current balance is 53.44

<script.py> output:
Hello John Doe. Your current balance is $53.44

<script.py> output:
Hello John Doe. Your current balance is $53.44.

In [1]:
```

Great work!

Solution

Submit

script.py

solution.py

```
1 s = "Strings are awesome!"
2 # Length should be 20
3 print("Length of s = %d" % len(s))
4
5 # First occurrence of "a" should be at index 8
6 print("The first occurrence of the letter a = %d"
  % s.index("a"))
7
8 # Number of a's should be 2
9 print("a occurs %d times" % s.count("a"))
10
11 # Slicing the string into bits
12 print("The first five characters are '%s'" % s[
  :5]) # Start to 5
13 print("The next five characters are '%s'" % s[5
  :10]) # 5 to 10
14 print("The last five characters are '%s'" % s[-5:
  :]) # 5 to end
15 print("The characters with odd index are '%s'" % s[
  1::2]) # All the characters with odd index are at
  index 1, 3, 5, 7, 9, 11, 13, 15, 17, 19
16 print("The characters that don't have an odd index are '%s'"
  % s[::2]) # All the characters not having an
  odd index are at index 0, 2, 4, 6, 8, 10, 12, 14, 16, 18
17 print("The first five characters in uppercase are '%s'"
  % s[:5].upper())
18 print("The last two characters in uppercase are '%s'"
  % s[-2:].upper())
19 print("The string is separated on words by spaces (split())
  and your string is '%s'" % s.split())
20 print("The string is separated on words by spaces (split())
  and your string is '%s'" % s.split())
```

IPython Shell

```
Length of s = 20
The first occurrence of the letter a = 8
a occurs 2 times
The first five characters are 'Strin'
The next five characters are 'gs ar'
The thirteenth character is 'a'
The characters with odd index are 'tig r wsm!'
The last five characters are 'some!'
String in uppercase: STRINGS ARE AWESOME!
String in lowercase: strings are awesome!
String starts with 'Str'. Good!
String ends with 'ome!'. Good!
Split the words of the string: ['Strings', 'are',
  'awesome!']
```

Great work!

Submit

script.py

```
1 # change this code
2 number = 104
3 second_number = 0
4 first_array = [1,3,2]
5 second_array = [1,2]
6
7* if number > 15:
8     print("1")
9
10* if first_array:
11     print("2")
12
13* if len(second_array) == 2:
14     print("3")
15
16* if len(first_array) + len(second_array) == 5:
```

IPython Shell

```
<script.py> output:
1
2
3
4
5
6

In [1]:
```

Great Work!

Solution

Submit

script.py

```
4 386, 462, 47, 418, 907, 344, 236, 375, 823,
5 566, 597, 978, 328, 615, 953, 345,
6 399, 162, 758, 219, 918, 237, 412, 566, 826,
7 248, 866, 950, 626, 949, 687, 217,
8 815, 67, 104, 58, 512, 24, 892, 894, 767, 553
9 , 81, 379, 843, 831, 445, 742, 717,
10 958, 609, 842, 451, 688, 753, 854, 685, 93,
11 857, 440, 380, 126, 721, 328, 753, 470,
12 743, 527
13 ]
14 for number in numbers:
15     if (number == 237):
16         break
17     if (number % 2 == 1):
18         continue
```

IPython Shell

```
592
236
942
386
462
418
344
236
566
978
328
162
758
918
```

Great work!

Solution

Submit

script.py

```

- strings as defined above
2 * def list_benefits():
3     return ["More organized code", "More readable
code", "Easier code reuse", "Allowing programmers
to share and connect code together"]
4
5 # Modify this function to concatenate to each
benefit - " is a benefit of functions!"
6 * def build_sentence(benefit):
7     return (benefit + | " is a benefit of
functions!")
8
9 * def name_the_benefits_of_functions():
10     list_of_benefits = list_benefits()
11 *     for benefit in list_of_benefits:
12         print(build_sentence(benefit))

```

IPython Shell

```

<script.py> output:
('More organized code', ' is a benefit of functions!')
('More readable code', ' is a benefit of functions!')
('Easier code reuse', ' is a benefit of functions!')
('Allowing programmers to share and connect code
together', ' is a benefit of functions!')

<script.py> output:
More organized code is a benefit of functions!
More readable code is a benefit of functions!
Easier code reuse is a benefit of functions!
Allowing programmers to share and connect code together
is a benefit of functions!

```

Nice work!

Solution

Submit

script.py

solution.py

```

- (self.name, self.color, self.kind, self.value)
9         return desc_str
10 # your code goes here
11 car1 = Vehicle()
12 car1.name = "Fer"
13 car1.kind = "convertible"
14 car1.color = "red"
15 car1.value = 60000.00
16
17 car2 = Vehicle()
18 car2.name = "Jump"
19 car2.kind = "van"
20 car2.color = "blue"
21 car2.value = 10000.00
22
23 # test code

```

IPython Shell

```

<script.py> output:
Fer is a red convertible worth $60000.00.
Jump is a blue van worth $10000.00.

In [1]:

```

Great job!

Submit

script.py

```

1 * phonebook = {
2     "John" : 938477566,
3     "Jack" : 938377264,
4     "Jill" : 947662781,
5     "Jake" : 938273443
6 }
7 # your code goes here
8
9 # testing code
10 * if "Jake" in phonebook:
11     print("Jake is listed in the phonebook.")
12 phonebook.pop("Jill")
13 * if "Jill" not in phonebook:
14     print("Jill is not listed in the phonebook.")

```

IPython Shell

```

<script.py> output:
Jill is not listed in the phonebook.

<script.py> output:
Jill is not listed in the phonebook.

<script.py> output:
Jill is not listed in the phonebook.

<script.py> output:
Jake is listed in the phonebook.
Jill is not listed in the phonebook.

```

Nice work!

Solution

Submit

script.py

solution.py

```

1 import re
2
3 # Your code goes here
4 find_members = []
5 * for member in dir(re):
6     if "find" in member:
7         find_members.append(member)
8
9 print(sorted(find_members))

```

IPython Shell

```

<script.py> output:
['findall', 'finditer']

In [1]:

```

Great work!

Submit

script.py

```

1 # write your code here
2 numbers = input().split()
3 print("sum is 0")

```

IPython Shell

In [1]:

Solution

Submit

script.py

```

1 # fill in this function
2 def fib():
3     a,b = 1,1
4     while 1:
5         yield a
6         a,b = b,a+b
7 import types
8 if type(fib()) == types.GeneratorType:
9     print("Good, The fib function is a generator.")
10
11 counter = 0
12 for n in fib():
13     print(n)
14     counter += 1
15 if counter == 10:
16     break

```

IPython Shell

```

<script.py> output:
Good, The fib function is a generator.
1
1
2
3
5
8
13
21
34
55

```

Good work!

Submit

script.py

```

1 numbers = [34.6, -203.4, 44.9, 68.3, -12.2, 44.6, 12.7]
2 newlist = [int(number) for number in numbers if number>0]
3 print(newlist)

```

IPython Shell

```

newlist = [number for number in numbers of number%2==0]
^
SyntaxError: invalid syntax
File "script.py", line 2
newlist = [int(number) for number in numbers of
number%2==0]
^
SyntaxError: invalid syntax

<script.py> output:
[]

<script.py> output:
[34, 44, 68, 44, 12]

```

Very nice!

Submit

script.py

```

1 l = [2,4,7,3,14,19]
2 for i in l:
3     fun = lambda x : (x%2)==0
4     print(fun(i))

```

IPython Shell

```

File "script.py", line 4
print(lambda(i))
^
SyntaxError: invalid syntax

<script.py> output:
True
True
False
False
True
False

In [1]:

```

Nice work!

Submit

script.py

```

1 def foo(a, b, c, *alte):
2     return len(alte)
3
4 def bar(a, b, c, *alte):
5     return alte["magicnumber"] == 7
6
7
8 # test code
9 if foo(1, 2, 3, 4) == 1:
10     print("Good.")
11 if foo(1, 2, 3, 4, 5) == 2:
12     print("Better.")
13 if bar(1, 2, 3, magicnumber=6) == False:
14     print("Great.")
15 if bar(1, 2, 3, magicnumber=7) == True:
16     print("Awesome!")

```

IPython Shell

```

<script.py> output:
Good.
Better.

Traceback (most recent call last):
  File "script.py", line 13, in <module>
    if bar(1, 2, 3, magicnumber=6) == False:
TypeError: bar() got an unexpected keyword argument
'magicnumber'

<script.py> output:
Good.
Better.
Great.
Awesome!

```

Great work!

Submit

script.py

```

1 # Exercise: make a regular expression that will
  match an email
2 import re
3 def test_email(your_pattern):
4     pattern = re.compile(your_pattern)
5     emails = ["john@example.com", "python
-list@python.org", "wha.t.lan?ug{}ly@email.com"]
6     for email in emails:
7         if not re.match(pattern, email):
8             print("You failed to match %s" % (email
9         ))
10    elif not your_pattern:
11        print("Forgot to enter a pattern!")
12    else:
13        print("Pass")
14 # Your pattern here!

```

IPython Shell

```

<script.py> output:
<_sre.SRE_Match object; span=(35, 39), match='[on]'\>
None
Forgot to enter a pattern!
Forgot to enter a pattern!
Forgot to enter a pattern!

<script.py> output:
Pass
Pass
Pass

In [1]:

```

Great work!

Submit

script.py

```

1 # Setup
2 actor = {"name": "John Cleese", "rank": "awesome"}
3
4 # Function to modify!!!
5 def get_last_name():
6
7     return actor["name"].split()[1]
8
9 # Test code
10 get_last_name()
11 print("All exceptions caught! Good job!")
12 print("The actor's last name is %s" % get_last_name
13     ())

```

IPython Shell

```

<script.py> output:
All exceptions caught! Good job!
The actor's last name is John Cleese

Traceback (most recent call last):
  File "script.py", line 10, in <module>
    get_last_name()
  File "script.py", line 7, in get_last_name
    return actor["name"].split(1)
TypeError: Can't convert 'int' object to str implicitly

<script.py> output:
All exceptions caught! Good job!
The actor's last name is Cleese

```

Great work!

Submit

script.py

```

1 a = set(["Jake", "John", "Eric"])
2 b = set(["John", "Jill"])
3 print(a-b)

```

IPython Shell

```

<script.py> output:
{'Eric', 'Jake'}

In [1]:

```

Nice work!

Solution

Submit

script.py

1 import json

2

3 # fix this function, so it adds the given name

4 # and salary pair to salaries_json, and return it

5 def add_employee(salaries_json, name, salary):

6 salaries = json.loads(salaries_json)

7 salaries[name] = salary

8

9 return json.dumps(salaries)

10

11 # test code

12 salaries = '{"Alfred" : 300, "Jane" : 400 }'

13 new_salaries = add_employee(salaries, "Me", 800)

14 decoded_salaries = json.loads(new_salaries)

15 print(decoded_salaries["Alfred"])

16 print(decoded_salaries["Jane"])

17

solution.py

IPython Shell

```
<script.py> output:
300
400
800

In [1]:
```

Great work!

Submit

solution.py

1 * Following is the exercise, function provided:

2 from functools import partial

3 def func(u, v, w, x):

4 return u*4 + v*3 + w*2 + x

5

6 p=partial(func,5,6,7)

7 print(p(8))

8

script.py

IPython Shell

```
Traceback (most recent call last):
  File "script.py", line 6, in <module>
    p=partial(func(x,5,6,7))
NameError: name 'x' is not defined

Traceback (most recent call last):
  File "script.py", line 6, in <module>
    p=partial(func(5,6,7))
TypeError: func() missing 1 required positional argument: 'x'

<script.py> output:
60

In [1]:
```

Good job!

Submit

script.py

1 # Use the help function to see what each function

2 # does.

3 # Delete this when you are done.

4 help(dir)

5 help(hasattr)

6 help(id)

7

8 # Define the Vehicle class.

9 class Vehicle:

10 name = ""

11 kind = "car"

12 color = ""

13 value = 100.00

14 def description(self):

15 desc_str = "%s is a %s worth %.2f." %

16 (self.name, self.color, self.kind, self.value)

solution.py

IPython Shell

```
Return the identity of an object.

This is guaranteed to be unique among
simultaneously existing objects.
(CPython uses the object's memory address.)

['__class__', '__delattr__', '__dict__', '__dir__',
 '__doc__', '__eq__', '__format__', '__ge__',
 '__getattr__', '__gt__', '__hash__',
 '__init__', '__le__', '__lt__', '__module__',
 '__ne__', '__new__', '__reduce__', '__reduce_ex__',
 '__repr__', '__setattr__', '__sizeof__', '__str__',
 '__subclasshook__', '__weakref__', 'color',
 'description', 'kind', 'name', 'value']
```

Very nice!

Submit

solution.py

1 def multiplier_of(n):

2 def multiplier(number):

3 return n*number

4 return multiplier

5 multiplywith5 = multiplier_of(5)

6 print(multiplywith5(9))

script.py

IPython Shell

```
Traceback (most recent call last):
  File "script.py", line 5, in <module>
    multiplywith5 = multiplier_of(5)
  File "script.py", line 4, in multiplier_of
    return number
NameError: name 'number' is not defined

<script.py> output:
45

In [1]:
```

Great work!

Submit

script.py

solution.py

```
6 *         else:
7             print("Bad Type")
8             return new_function
9             return check
10
11 @type_check(int)
12 * def times2(num):
13     return num*2
14
15 print(times2(2))
16 times2('Not A Number')
17
18 @type_check(str)
19 * def first_letter(word):
20     return word[0]
21
```

IPython Shell

<script.py> output:

```
4
Bad Type
H
Bad Type

In [1]: |
```

Good job!

Submit

script.py

solution.py

```
2 from functools import reduce
3
4 my_floats = [4.35, 6.09, 3.25, 9.77, 2.16, 8.88,
5 4.59]
6 my_names = ["olumide", "akinremi", "josiah",
7 "temidayo", "omoseun"]
8 my_numbers = [4, 6, 9, 23, 5]
9
10 map_result = list(map(lambda x: round(x ** 2, 3),
11 my_floats))
12 filter_result = list(filter(lambda name: len(name)
13 <= 7, my_names))
14 reduce_result = reduce(lambda num1, num2: num1 *
15 num2, my_numbers)
16
17 print(map_result)
```

IPython Shell

<script.py> output:

```
[18.922, 37.088, 10.562, 95.453, 4.666, 78.854, 21.068]
['olumide', 'josiah', 'omoseun']
24840

In [1]: |
```

Congrats! Nice work.

Submit

BIBLIOGRAFIE

1. [learnpython.org](https://www.learnpython.org/)
2. [W3schools.com](https://www.w3schools.com/)